

IMPLEMENTASI DAN DEPLOYMENT MICROSERVICE PADA KUBERNETES CLUSTER MENGGUNAKAN MINIKUBE



Disusun Oleh :

Ariski Myardi

1103220019

TK-46-04

Pendahuluan

Di era teknologi digital pada masa ini, kebutuhan akan sistem yang mampu menangani pertumbuhan data dan pengguna secara efisien menjadi sangat penting. Sistem monolitik tradisional sering kali tidak dapat memenuhi kebutuhan ini karena keterbatasan skalabilitas, fleksibilitas, dan kecepatan pengembangan. Oleh karena itu, arsitektur layanan mikro telah muncul sebagai solusi yang sangat dicari dalam pengembangan perangkat lunak modern.

Layanan mikro memungkinkan pengembangan aplikasi yang terdiri dari layanan kecil dan independen. Layanan ini dapat dikembangkan, diuji, dan diterapkan secara terpisah tanpa mengganggu keseluruhan sistem. Namun, tantangan utama dalam implementasi layanan mikro adalah mengelola orkestrasi layanan yang kompleks, terutama ketika jumlah layanan meningkat.

Kubernetes, platform orkestrasi kontainer, dirancang untuk mengatasi tantangan ini. Dengan fitur-fitur seperti penyeimbangan beban, penskalaan otomatis, dan pemulihan mandiri, Kubernetes menyediakan solusi yang kuat untuk mengelola lingkungan layanan mikro. Minikube, sebagai versi lokal Kubernetes, memungkinkan pengembang untuk mensimulasikan kluster Kubernetes di lingkungan lokal untuk tujuan pengujian dan pengembangan.

Proyek ini bertujuan untuk menunjukkan bagaimana layanan mikro dapat diimplementasikan dalam kluster Kubernetes menggunakan Minikube. Selain itu, proyek ini juga bertujuan untuk memahami tantangan yang muncul selama proses penerapan dan menemukan solusi yang tepat untuk mengatasinya.

Arsitektur Sistem

Arsitektur sistem terdiri dari beberapa komponen utama sebagai berikut:

- **Minikube** yang berfungsi sebagai cluster Kubernetes lokal.
- **Docker** yang digunakan untuk menjalankan Minikube dan membuat image container dari aplikasi.
- **Kubernetes Deployment** yang bertugas untuk mengelola replikasi pod.
- **Kubernetes Service** yang mengatur komunikasi antar pod serta menyediakan akses bagi pengguna.

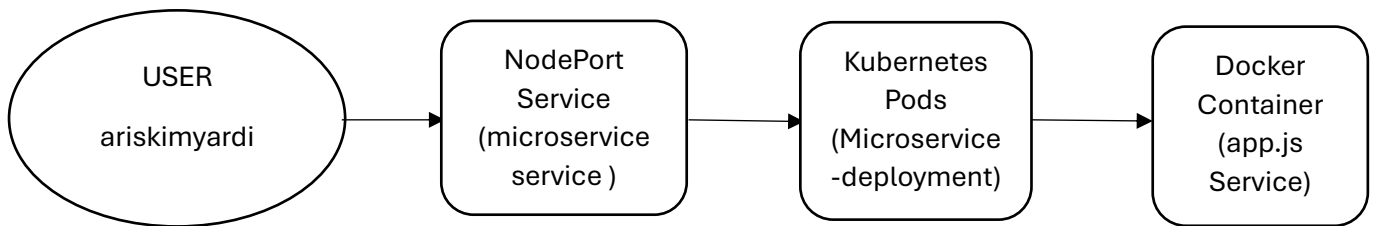


Diagram Arsitektur Kubernetes

Langkah Deployment

1. Instalasi Kubernetes menggunakan Minikube
 - Instalasi Kubectl
 - Instalasi Kubernetes dengan Minikube dengan menggunakan perintah “curl -LO <https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64>” dan “sudo install minikube-linux-amd64 /usr /usr/local/bin/minikube”
 - Menjalankan Minikube menggunakan perintah “minikube start”
2. Membangun Layanan
 - Membuat layanan app.js
 - Inisialisasi dengan perintah “npm init -y”
3. Membuat Docker Image
 - Menulis Dockerfile

```
GNU nano 7.2 Dockerfile
# Choose your image sebagai base image
FROM node:16

# Set direktori kerja di dalam container
WORKDIR /usr/src/app

# Salin file package.json dan package-lock.json (jika ada)
COPY package*.json ./

# Install dependencies
RUN npm install

# Salin seluruh file ke dalam container
COPY . .

# Ekspose port aplikasi
EXPOSE 3000

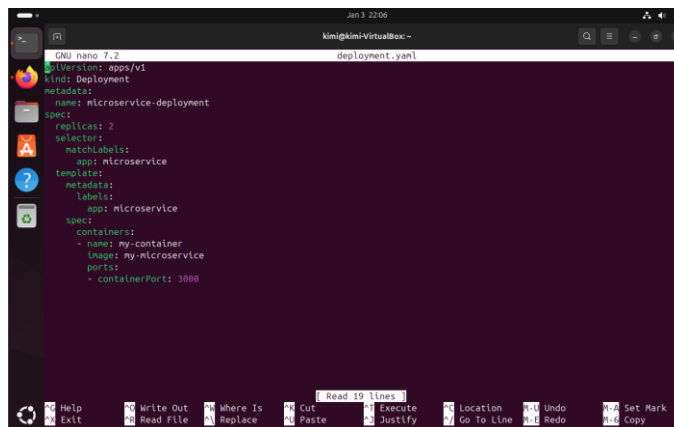
# Jalankan aplikasi
CMD ["node", "app.js"]
```

The screenshot shows a terminal window with a nano editor editing a Dockerfile. The Dockerfile contains instructions for building a container image based on Node.js 16, setting the working directory to /usr/src/app, copying package files, installing dependencies, and running the application on port 3000.

- Membangun image dengan perintah “docker build -t my-microservice”

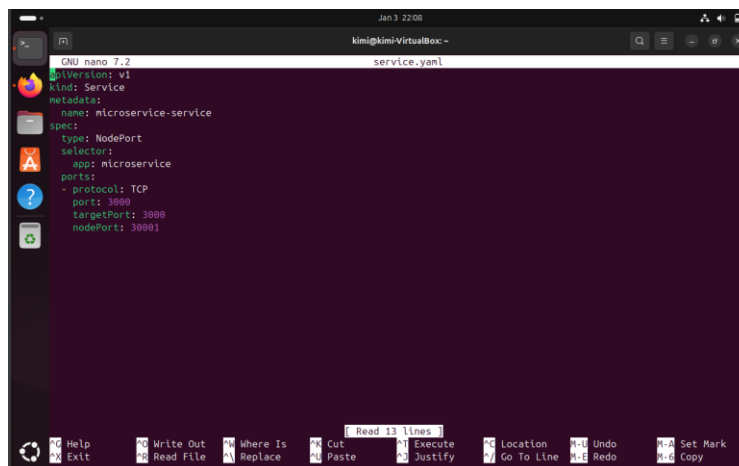
- Menjalankan container lokal dengan perintah “docker run -p 3000:3000 my-microservice”
4. Push Docker image ke Docker Hub
- Login Ke Docker Hub menggunakan perintah “docker login”
 - Push image dengan perintah “docker push <username>/my-microservice”
5. Deployment ke Kubernetes
- Membuat file manifest “deployment.yaml” dan “service.yaml”

deployment.yaml



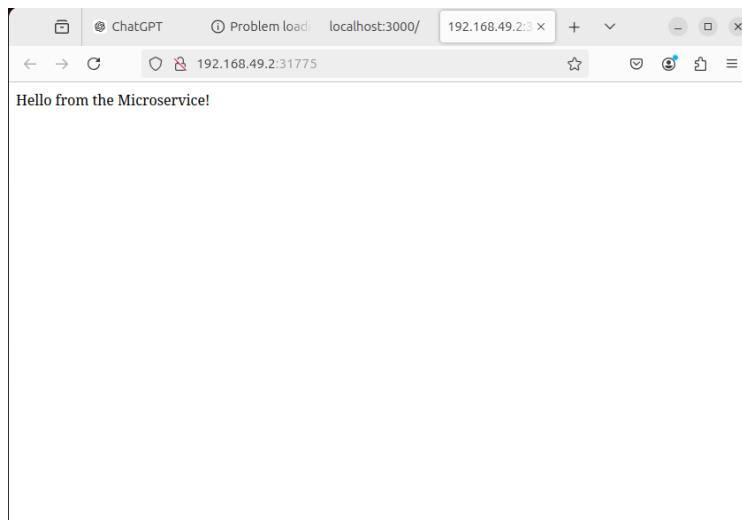
```
GNU nano 7.2 deployment.yaml
kind: Deployment
metadata:
  name: microservice-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: microservice
  template:
    metadata:
      labels:
        app: microservice
    spec:
      containers:
      - name: my-container
        image: my-microservice
        ports:
        - containerPort: 3000
```

service.yaml



```
GNU nano 7.2 service.yaml
kind: Service
metadata:
  name: microservice-service
spec:
  type: NodePort
  selector:
    app: microservice
  ports:
  - protocol: TCP
    port: 3000
    targetPort: 3000
    nodePort: 30001
```

- Deploy dengan perintah “`kubectl apply -f deployment.yaml`” dan “`kubectl apply -f service.yaml`”
6. Verifikasi Deployment
- Verifikasi Deployment dengan perintah “`kubectl get deployments`” dan “`kubectl get services`”
7. Akses pada Microservice
- Dapatkan IP Minikube dengan perintah “`minikube ip`”
 - Akses aplikasi melalui “`http://<minikube_ip>:30001`”.
 - Ip saya `http://192.168.49.2:30001/`



Hasil Implementasi

Berikut merupakan hasil implementasi yang berhasil dilakukan :

- Menverifikasi jumlah pada pods dengan perintah “`kubectl get pods`” dan memastikan semua pods dalam keadaan “Running”

```
kimi@kimi-VirtualBox:~/microservice$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
automatic-door-deployment-74bd77fc8b-xgn4d   1/1     Running   1 (36s ago)   104m
microservice-deployment-596d87b565-lc8dv     1/1     Running   1 (36s ago)   72m
kimi@kimi-VirtualBox:~/microservice$
```

```
kimi@kimi-VirtualBox:~/microservice$ minikube ip
192.168.49.2
```

Analisis Kendala dan Solusi

Salah satu kendala yang sering ditemui saat menggunakan Kubernetes adalah kesalahan konfigurasi port pada file manifest, seperti `deployment.yaml` dan `service.yaml`. Kesalahan ini bisa menyebabkan aplikasi tidak dapat dijalankan dengan benar, karena port yang digunakan tidak sesuai atau salah konfigurasi. Contohnya, jika port yang didefinisikan dalam `deployment.yaml` pada bagian `containerPort` tidak sama dengan port yang ada di `service.yaml`, maka service tidak akan bisa mengakses container yang tepat. Selain itu, port yang dipilih juga bisa berbenturan dengan port lain yang sudah digunakan oleh aplikasi atau service lain dalam cluster.

Untuk mengatasi hal ini, pastikan konfigurasi port pada kedua file tersebut sesuai. Pertama, periksa bahwa port yang digunakan dalam `deployment.yaml` pada bagian `containerPort` sudah sesuai dengan port yang digunakan oleh aplikasi. Selanjutnya, buka `service.yaml` dan pastikan port yang ditentukan di sana cocok dengan port yang ada di container. Jika keduanya tidak sesuai, service tidak akan bisa terhubung dengan aplikasi yang benar. Jangan lupa juga untuk memastikan port yang dipilih tidak berbenturan dengan port lain dalam cluster. Setelah melakukan perubahan, terapkan dengan perintah `kubectl apply -f deployment.yaml` dan `kubectl apply -f service.yaml`. Terakhir, pastikan service dapat terhubung dengan aplikasi dengan memeriksa status pod dan service menggunakan perintah `kubectl get pods` dan `kubectl get svc`. Dengan memastikan konfigurasi port sudah benar, masalah koneksi antar komponen dalam cluster Kubernetes bisa dihindari.

- | | |
|------------|--|
| 1. Kendala | : Salah Mengkonfigurasi port pada file manifest |
| Solusi | : Mencoba Kembali port pada “ <code>deployment.yaml</code> ” dan “ <code>service.yaml</code> ” |

Kesimpulan dan Rekomendasi

Proyek ini berhasil menunjukkan bagaimana proses deployment aplikasi berbasis microservices dapat dijalankan dengan baik di dalam cluster Kubernetes. Penggunaan Minikube mempermudah pengelolaan lingkungan lokal, memungkinkan pengembangan dan pengujian aplikasi yang lebih efisien. Untuk pengembangan lebih lanjut, ada beberapa langkah yang bisa diambil untuk meningkatkan sistem ini. Langkah pertama adalah menambahkan fitur load balancing dengan memanfaatkan Ingress untuk memastikan distribusi lalu lintas yang lebih merata antara pod, sehingga meningkatkan ketersediaan dan performa aplikasi. Selanjutnya, sistem bisa diintegrasikan dengan alat pemantauan seperti Grafana atau Prometheus untuk memantau kinerja aplikasi dan cluster secara real-time. Hal ini akan memudahkan deteksi masalah dan analisis metrik operasional. Terakhir, pengujian performa secara menyeluruh bisa dilakukan menggunakan alat seperti Apache JMeter untuk mengukur sejauh mana sistem dapat menangani beban tinggi dan mengidentifikasi potensi bottleneck. Langkah-langkah ini akan membantu meningkatkan stabilitas, skalabilitas, dan keandalan aplikasi dalam lingkungan produksi.

DEMO TEST

Pada demo test akan dilakukan beberapa test akhir untuk memastikan bahwa semua komponen berjalan baik. Berikut adalah langkah-langkah yang akan dilakukan:

- Verifikasi jumlah pods dan pastikan semua pods dalam status “Running”

```
kimi@kimi-VirtualBox:~/microservice$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
automatic-door-deployment-74bd77fc8b-xgn4d	1/1	Running	1 (36s ago)	104m
microservice-deployment-596d87b565-lc8dv	1/1	Running	1 (36s ago)	72m